

ASSIGNMENT # 02

Name: Nooran Ishtiaq

Roll no : 22i-2010

Section : DS-B

MLOps Pipeline using Apache Airflow and MLflow for Titanic Survival Prediction

1. Introduction

Machine Learning pipelines in real-world systems require automation, monitoring, and reproducibility. Traditional machine learning workflows often involve multiple manual steps such as data ingestion, preprocessing, training, evaluation, and deployment. Managing these steps manually increases the chances of errors and makes it difficult to reproduce results.

To solve this problem, modern MLOps tools such as Apache Airflow and MLflow are widely used. Apache Airflow is used for workflow orchestration and scheduling, while MLflow is used for experiment tracking, model management, and reproducibility.

In this project, an end-to-end machine learning pipeline was designed using Airflow to orchestrate tasks and MLflow to track experiments and manage models. The pipeline uses the Titanic Dataset (Kaggle Titanic Survival Dataset) to predict passenger survival based on demographic and travel features.

The pipeline includes data ingestion, validation, preprocessing, feature engineering, model training, evaluation, and model registration. The workflow is automated using an Airflow DAG, ensuring reliability, modularity, and scalability.

2. System Architecture (Airflow + MLflow Interaction)

The architecture of this system integrates workflow orchestration with experiment tracking.

Airflow acts as the pipeline orchestrator. Each step of the machine learning process is implemented as a task inside a Directed Acyclic Graph (DAG). Airflow manages task execution order, handles failures, and allows parallel execution of tasks.

MLflow is integrated within the training and evaluation tasks. During model training, MLflow records parameters, metrics, and artifacts. After evaluation, if the model meets the required performance threshold, it is registered in the MLflow Model Registry.

The interaction flow is as follows:

1. Airflow DAG triggers the pipeline execution.

2. Data ingestion loads the dataset.
3. Data validation ensures dataset quality.
4. Data preprocessing tasks run in parallel.
5. Encoded features are used to train a machine learning model.
6. MLflow logs training parameters, dataset size, and model artifacts.
7. Model evaluation computes metrics.
8. Based on accuracy, branching logic decides whether to register or reject the model.
9. Approved models are stored in the MLflow Model Registry.

This architecture ensures automation, reproducibility, and traceability of machine learning experiments.

3. DAG Structure and Task Dependency Explanation

The workflow is implemented using an Airflow DAG called **mlops_airflow_mlflow_pipeline**. A Directed Acyclic Graph ensures that tasks run in a defined order without cyclic dependencies.

The pipeline contains the following tasks:

3.1 Data Ingestion

The pipeline begins by loading the Titanic dataset from a CSV file. The dataset shape and missing values are logged for monitoring purposes. The dataset path is shared with downstream tasks using Airflow XCom.

3.2 Data Validation

The validation task checks the percentage of missing values in the **Age** and **Embarked** columns. If the missing value percentage exceeds 30%, the task raises an exception. This ensures that the model is trained only on acceptable data quality. To demonstrate Airflow's retry mechanism, an intentional failure was introduced in the validation step. Airflow automatically retries the task based on the retry configuration defined in the DAG.

3.3 Parallel Data Processing

After validation, two preprocessing tasks run in parallel:

- Handling missing values in Age and Embarked columns.
- Feature engineering by creating new features such as **FamilySize** and **IsAlone**.

Parallel execution reduces pipeline execution time and demonstrates Airflow's ability to manage concurrent tasks.

3.4 Data Encoding

Categorical variables such as **Sex** and **Embarked** are encoded into numerical values so they can be used by machine learning algorithms. Irrelevant columns such as PassengerId, Name, and Ticket are removed from the dataset.

3.5 Model Training

A machine learning model is trained using Logistic Regression or Random Forest. During training, MLflow logs:

- Model type
- Hyperparameters
- Dataset size
- Model artifact

This allows reproducibility and comparison across different runs.

3.6 Model Evaluation

After training, the model is evaluated using the following metrics:

- Accuracy
- Precision
- Recall
- F1 Score

These metrics are logged in MLflow for experiment comparison. The accuracy value is also pushed using XCom for use in branching logic.

3.7 Branching Logic

Airflow's BranchPythonOperator is used to implement conditional logic. If the model accuracy is greater than or equal to 0.80, the model is approved and sent for registration. Otherwise, the model is rejected and the rejection reason is logged.

3.8 Model Registration

Approved models are registered in the MLflow Model Registry. This allows versioning and easy deployment of machine learning models.

The DAG structure ensures clear task dependencies and prevents cyclic execution paths.

4. Experiment Comparison Analysis

To analyze model performance, the pipeline was executed three times with different hyperparameters. Each run was tracked using MLflow experiments.

The runs were compared using the MLflow UI, which provides visualization of parameters and metrics across experiments.

Example comparison:

Run 1

Model: Logistic Regression

Accuracy: 0.79

Run 2

Model: Random Forest (100 estimators)

Accuracy: 0.80

Run 3

Model: Random Forest (200 estimators)

Accuracy: 0.83

From the comparison, Run 3 achieved the highest accuracy and was therefore selected as the best-performing model. The MLflow interface allowed easy visualization of performance metrics and comparison between runs.

This experiment tracking capability is essential in real-world machine learning systems where multiple models must be evaluated and compared before deployment.

5. Failure Handling and Retry Mechanism

Airflow provides built-in mechanisms for fault tolerance. In this project, retry functionality was demonstrated during the data validation stage.

The validation task was configured with retry parameters such as the number of retries and delay between retries. If the validation task fails due to excessive missing values, Airflow automatically retries the task.

The retry attempts can be observed in the Airflow UI logs, where each retry attempt is recorded with timestamps and error messages. This functionality ensures that temporary failures do not stop the entire pipeline and improves system reliability.

The retry evidence was captured in the Airflow task logs and included in the project screenshots.

6. Reflection on Production Deployment

- While this project demonstrates a local development setup, deploying this pipeline in production would require additional considerations.
- First, the Airflow scheduler and workers should be deployed on scalable infrastructure such as Kubernetes or cloud-based managed services. This would allow the pipeline to handle larger datasets and higher workloads.
- Second, remote logging and artifact storage should be configured using cloud storage services such as AWS S3 or Google Cloud Storage to ensure persistence and scalability.
- Third, monitoring and alerting systems should be integrated to notify engineers in case of pipeline failures.
- Finally, the MLflow Model Registry can be integrated with CI/CD pipelines to automatically deploy approved models into production environments such as REST APIs or batch prediction systems.
- By implementing these improvements, the pipeline can be transformed into a fully production-ready MLOps workflow.

7. Conclusion

This project demonstrates the implementation of an end-to-end machine learning pipeline using Apache Airflow and MLflow. Airflow was used to orchestrate the workflow and manage task dependencies, while MLflow was used for experiment tracking and model management. The pipeline successfully automated data ingestion, validation, preprocessing, model training, evaluation, and model registration. Parallel processing and branching logic improved efficiency and flexibility of the workflow. Experiment tracking allowed comparison between multiple model runs, enabling selection of the best-performing model. Additionally, failure handling and retry mechanisms demonstrated the robustness of the pipeline. Overall, this project illustrates how modern MLOps tools can streamline machine learning workflows and make them more reliable and reproducible.